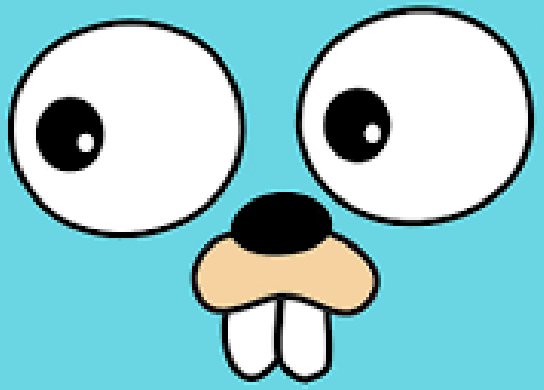

File Handling and I/O

Discussion Points

- Reading from and writing to files
- Working with *JSON* and *XML*
- Using the *io* and *os* packages

What is Open from files?

Files in Go are represented by pointers to objects of type **os.File**, also called file handles.

```
inputFile, inputError := os.Open("sessions/13/notes/input.txt")
defer inputFile.Close()
```

What is Reading from files?

Using **bufio's** reader makes work with convenient high-level string objects, completely insulated from the raw bytes which represent the text on disk.

```
inputReader := bufio.NewReader(inputFile)
```

Reading contents of an entire file in a string

```
inputFile := "sessions/13/notes/input.txt"
buf, err := os.ReadFile(inputFile)
```

Don't use ReadFile for big files

When to Use Each?

Use bufio.NewReader when:
You need to process the file in parts (e.g., line-by-line).
The file size is large, and loading it entirely into memory isn't feasible.
Memory efficiency is important for your application.

Use os.ReadFile when:
The file size is small or manageable, and you want a quick, straightforward way to read it.
You need the whole file content at once for processing.
Simplicity is preferred over memory efficiency (e.g., for configuration files or small data files).

Reading from and writing to files

How to Create files?

Files in Go creates using ***os*** package and need to provide access writes and permissions to the current file

The **os.OpenFile** is a function that allows more control over the opening mode of a file. We see that the OpenFile function takes a filename, one or more flags and the file permissions to use. The following flags are commonly used:

- os.O_RDONLY: the read flag for read-only access
- os.O_WRONLY: the write flag for write-only access
- os.O_RDWR: the flag that provides both read and write access
- os.O_CREATE: the create flag to create the file if it doesn't exist
- os.O_TRUNC: the truncate flag to truncate to size 0 if the file already exists

When reading, the file permissions are ignored so we can use a value of 0. When writing, we use the standard Unix file permissions of 0666 (even on Windows).

```
outputFile, outputError := os.OpenFile(filename, os.O_WRONLY|os.O_CREATE, 0666)
if outputError != nil {
    fmt.Printf("An error occurred with file creation\n")
    return
}
defer outputFile.Close()
```

How to Write to files?

We are using **bufio.NewWriter()** to write by-line data to given file

```
outputWriter := bufio.NewWriter(outputFile)
outputString := "hello world!\n"
outputWriter.WriteString(outputString)
outputWriter.Flush()
```

How to Write to files?

We are using **bufio.NewWriter()** to write by-line data to given file

```
outputWriter := bufio.NewWriter(outputFile)
outputString := "hello world!\n"
outputWriter.WriteString(outputString)
outputWriter.Flush()
```

How to Write top or end to files?

It really depends on how we opened file

```
f, err := os.OpenFile("sessions/13/notes/writing/output.dat", os.O_CREATE|os.O_WRONLY, 0)
```

```
f, err := os.OpenFile("sessions/13/notes/writing/output.dat", os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0)
```

The JSON Data Format?

Data in JSON formats can be transmitted over a network. In this way, the data is platform-independent and can be used as input/output in all kinds of applications, no matter what the programming language or the operating system is.

The following terms are common here:

data structure to special format string = marshaling or encoding (before transmission at the sender or source)
special format string to data structure = unmarshaling or decoding (after transmission at the receiver or destination)

Marshaling is used to convert in-memory data to the special format (data -> string), and vice versa for **unmarshaling** (string -> data structure).

Working with JSON and XML

How to Marshaling JSON?

The json.Marshal() encodes the data into the following json-text

Encoding Function: json.Marshal() converts Go data to JSON format. json.NewEncoder().Encode() will encode, and write the encoded json data to writer

Default Type Mappings:
bool → JSON Boolean
float64 → JSON number
string → JSON string
nil → JSON null

Limitations:
Only map[string]T can be used for JSON objects (keys must be strings).
Unsupported types: channels, complex numbers, functions.
Cyclic structures are not allowed (will cause an infinite loop).
Pointers are converted to their values (or null if nil).
Only exported fields in structs are included in JSON output (due to reflection).

How to UnMarshaling JSON?

If we know beforehand the semantics of the json-data, we can then define an appropriate struct and unmarshal into it

The json package also allows to encode and decode json, having fields names different from the ones in a struct

Working with XML?

XML is a markup language that sets some rules to encode data in both human and machine-readable format. It stands for eXtensible Markup Language.

As a json formats, xml also requires marshaling and unmarshaling implementation. Programming requires some object behaviour and mapping, so XML format is not an object, just some structured text. Using **xml** package we can quickly implement and encode/decode xml to objects

How to Marshal/UnMarshal XML?

Like the json package xml contains a Marshal() and an UnMarshal() function to encode and decode data to and from XML. In the same way as with JSON, XML-data can be marshaled or un-marshaled to/from structs.

Remark: Use Unmarshal to deserialize a short text you already have in memory (e.g., some user input) and NewDecoder when you are reading from a stream (e.g., from a file).

Thank you.
Questions?

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- Albert Einstein